

XPU-RT: Feedback-Driven Realtime Deployment of Robotic Workloads on Heterogeneous SoCs

Shengjun Kris Dong
University of California, Berkeley
USA
krisdong@berkeley.edu

Dima Nikiforov
University of California, Berkeley
USA
vnikiforov@berkeley.edu

Agustin N. Coppari Hollmann
University of California, Berkeley
USA
copparihollmann@berkeley.edu

Ailsa Sun
University of California, Berkeley
USA
23suna18@berkeley.edu

Yakun Sophia Shao
University of California, Berkeley
USA
ysshao@berkeley.edu

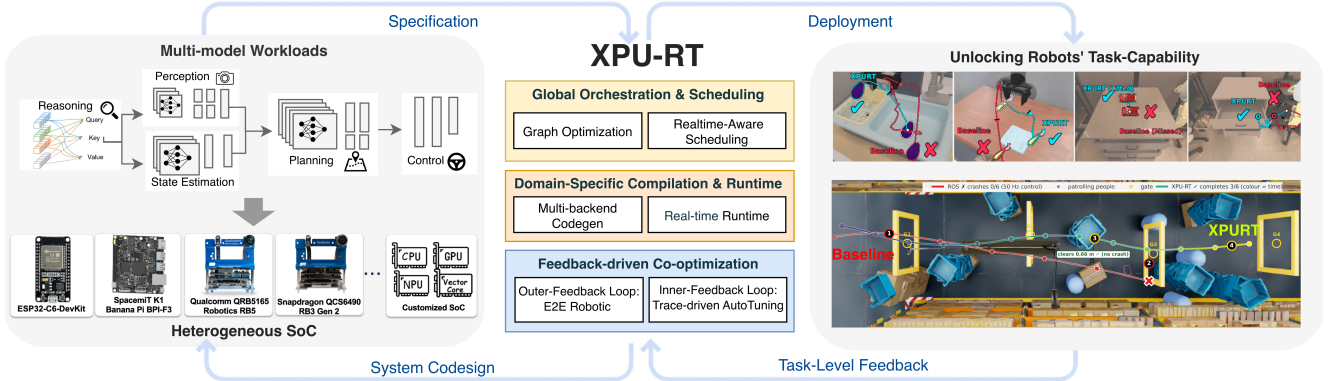


Figure 1. XPU-RT deploys real-time multi-model robotic workloads onto heterogeneous SoCs. XPU-RT extrapolates specifications from user-defined workloads, generates an optimized binary to the target platforms with mapping and binding encoded, and fine tunes system knobs based on the feedback from the robot’s measurements that drives the next solve.

Abstract

As robotic workloads scale to multiple models, robots must meet real-time deadlines despite tight onboard compute. Failure to meet performance requirements can lead to task failures, collisions and physical damage. Existing solutions rely on over-provisioning, pinning each model to one engine and reserving for a worst case that rarely arrives, which leaves compute in idle. Other approaches leverage priority-based preemption or timer interrupts, but neither carries over to a heterogeneous SoC: not every accelerator is preemptible, and runtime contention under a real-time constraint cannot be guaranteed. Robotic deployment must therefore coordinate compilation, placement, and scheduling across the whole workload and evaluate the resulting behavior in closed loop.

We present XPU-RT, a full-stack framework for deploying real-time multi-model robotic workloads across commercial SoCs and custom RTL designs. XPU-RT carries a workload from specification through multi-backend implementation generation, heterogeneous scheduling, and runtime execution, while closing feedback from measured hardware behavior back into both scheduling and compilation, and from robot-level outcomes back into deployment decisions. XPU-RT improves end-to-end performance across diverse vendor

SDKs and robot morphologies. On a Qualcomm robotics SoC it raises the action rate of a manipulation policy by 4.7× over the vendor runtime and improves task success by 1.27×. Generalizing to a drone-racing workload on a commercial RISC-V board, it sustains 2.5× the control rate the Robotic Operating System (ROS 2) reaches with zero deadline misses, clearing a requirement the baseline misses outright, and in closed-loop flight has a 50% success rate compared to 0% in a ROS 2 loop.

1 Introduction

Modern robots execute perception, planning, control, and reasoning models concurrently on a shared compute platform. These workloads have different rates, dependencies, and deadlines, yet contend for the same compute, memory, and I/O resources [3, 6, 18]. At the same time, robot compute has evolved from simple embedded processors into heterogeneous SoCs combining CPUs, vector cores, GPUs, and specialized accelerators, each with different compilation, memory, and runtime interfaces [4, 24, 33, 40]. Deployment is therefore no longer simply a matter of optimizing each

model independently: the system must decide which implementation to use, where it should execute and when it should run.

Robotics makes this orchestration problem unusually heterogeneous. Robots differ widely in morphology, sensing, form factor, and operating environment, which in turn changes both the workload and the platform on which it must run [17, 35]. Size, weight, and power (SWaP) constraints amplify this diversity: they do not merely cap the amount of available compute, but often force a deployment onto a particular class of processor, accelerator, memory system, or custom SoC chosen for that robot’s physical envelope [8, 16, 36]. In less SWaP-constrained domains, workloads can converge around a relatively standard set of compute platforms; robotic systems instead span microcontrollers, embedded SoCs, custom accelerators, and higher-end compute modules. The software stack is correspondingly fragmented across vendor SDKs, toolchains, and runtime interfaces [10].

Finally, robots operate in a closed loop, so component-level performance is only a proxy for the quantity that ultimately matters. The effect of a model’s latency depends on where it sits in the perception–planning–control loop and how often its outputs are consumed. A computationally inexpensive control model running too slowly can destabilize a robot even while more expensive perception models meet their own performance targets [8, 23]. Consequently, the best deployment depends jointly on the robot, its workload, the hardware available under its physical constraints, and the resulting closed-loop behavior.

Robotic deployment is therefore a system-level optimization problem. Existing compilers, schedulers, and runtimes address important pieces of this problem, but typically optimize within a fixed implementation space or evaluate component-level proxies rather than closing the loop through the deployed robot.

To address this problem, we develop XPU-RT, a full-stack deployment framework for real-time robotic workloads on heterogeneous SoCs. XPU-RT takes the entire robot workload from its models, rates, dependencies, deadlines, and available hardware and carries it through graph transformation, multi-backend implementation generation, heterogeneous scheduling, and runtime execution under a common set of abstractions. Crucially, deployment is not the endpoint: measured hardware behavior feeds back into scheduling and compilation, allowing XPU-RT to revise both placement and the available implementation space, while closed-loop robot outcomes feed back into deployment decisions. This lets the same framework exploit platform-specific opportunities from multi-hart sharding on RISC-V to accelerator placement and pipelining on commercial SoCs without changing the application-level interface.

Our evaluation shows that these feedback loops change both what can be scheduled and what the robot ultimately achieves. On SpaceMiT K1, feedback reduces the optimal

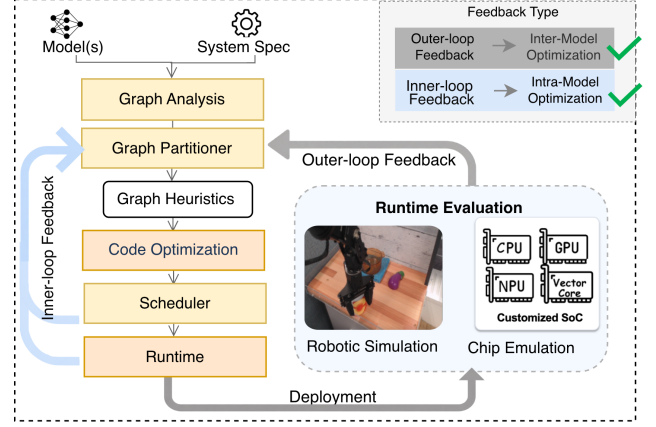


Figure 2. A flow chat representing XPU-RT’s 2-level hierarchical feedback loops. XPU-RT intakes feedback from hardware measurements and robot behavior to optimize the compilation and scheduling.

critical-model response from 8.00 ms to 4.89 ms, *below the lower bound of the original implementation graph*, showing that the gain comes from changing the available implementation space rather than searching it more effectively. On Qualcomm QRB5165, heterogeneous placement and pipelining increase manipulation-policy action rate 4.7× over the vendor runtime and substantially improve task success. In closed-loop flight, XPU-RT sustains a 100 Hz control loop that the ROS 2 baseline cannot meet, completing 3/6 matched flights versus 0/6.

XPU-RT’s contributions are:

- **A unified full-stack abstraction for heterogeneous robotic deployment.** XPU-RT represents multi-model workloads, timing constraints, implementations, hardware capabilities, and schedules through shared interfaces spanning commercial SoCs and RTL platforms.
- **Feedback-driven cross-stack optimization.** Hardware measurements refine scheduling costs and can trigger graph and implementation changes, while robot-level evaluation feeds task outcomes back into deployment decisions rather than relying only on component-level latency.
- **A portable runtime and evaluation framework demonstrated end to end.** XPU-RT generates schedule-driven runtimes across Linux, Zephyr, vendor SDKs, and pre-silicon platforms, and we evaluate the resulting deployments from per-dispatch timing through manipulation and autonomous flight.
- **An open-source¹ automated, modular framework** that is adaptable across hardware targets, robotic platforms, algorithms, and deployment scenarios, enabling agile specialization to highly optimized robot deployments.

¹<https://github.com/ucb-bar/XPU-RT>.

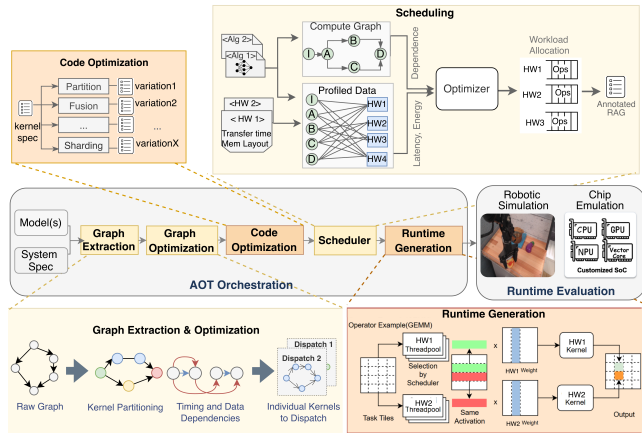


Figure 3. The XPU-RT end-to-end toolchain, highlighting the workflow from initial application specification to executable binary.

2 Background and Motivation

Meeting a robot’s deadlines on a heterogeneous SoC is an orchestration problem: resource allocation, scheduling, and code generation have to be co-optimized together, against the hardware actually present. A system that does this needs three components, and we organize the prior work by them.

2.1 Global Orchestration and Scheduling

Prior work spans both implementation generation and system-level scheduling. Halide separates algorithms from execution schedules [29], while TVM, Ansor, FlexTensor, and Exo2 search or expose increasingly rich implementation spaces across heterogeneous targets [4, 12, 38, 39]; IREE provides an end-to-end compiler/runtime path [33], and libraries such as cuDNN supply optimized kernels [5]. These systems construct efficient implementations, but do not by themselves orchestrate a robot’s concurrent, multi-rate workload.

It’s necessary to analyze or construct schedules under system constraints. This includes mixed-criticality and ROS 2 response-time analysis [3, 34], heterogeneous CPS and multi-XPU scheduling [11, 18, 24], dynamic multi-model adaptation [13], contention-aware mapping [6], and priority/preemption mechanisms [31, 40]. Hardware/software co-design expands the space further through workload-aware SoC exploration [30, 32], while robotic systems incorporate runtime adaptation, physical objectives, and mission-level evaluation [1, 2, 14, 15, 21, 22, 37].

XPU-RT builds on these capabilities rather than introducing a new solver: it uses standard optimization engines such as CP-SAT and MILP [19, 28]. Its distinction is the scope of the optimization loop: one schedule spans the full multi-model workload, measured execution can feed back through scheduling into compilation, and closed-loop robot behavior feeds back into deployment decisions.

2.2 Domain-Specific Compilation and Runtime

Existing ML deployment stacks emphasize portability across targets. ONNX and ONNX Runtime represent and partition portable graphs [25, 26], while ExecuTorch lowers PyTorch programs to compact serialized plans for embedded execution [20, 27]; runtimes ultimately rely on target backends and libraries such as XNNPACK [9]. This abstraction is valuable for general deployment, but leaves many target-specific choices—including layouts, implementations, sharding, and schedule structure—to separate backend mechanisms. XPU-RT instead exposes these choices through a common robot-oriented compilation and runtime interface, allowing them to participate in system-level scheduling and feedback.

2.3 Feedback-Driven Co-optimization

Measurement already drives many optimization systems: autotuners replace analytic estimates with target measurements [4, 38, 39], RoSE enables pre-silicon robot/SoC co-simulation [23], MAVBench evaluates compute through mission outcomes [1], and robotic co-design systems optimize hardware or software against physical objectives [8, 14, 21, 22, 37]. What is less common is allowing deployed measurements to propagate far enough back through the stack to change the available implementation space itself. XPU-RT closes the path: hardware feedback can trigger both re-scheduling and new compiler choices, while robot-level feedback determines which resulting operating point is useful.

2.4 What Prior Work Lacks and XPU-RT Supplies

Prior work addresses many pieces of robotic deployment well in isolation, but four gaps remain when those pieces must operate as one system.

First, existing systems optimize proxies rather than the robotic objective. Tensor compilers and autotuners minimize kernel or subgraph latency, while list schedulers minimize makespan. Neither directly bounds the age of the data behind an actuation, so fast kernels and a short schedule can still miss a periodic control window. Our solver sweep illustrates this: deadline-blind HEFT remains competitive on makespan while accumulating more than 10^3 deadline misses (§5.6). XPU-RT instead schedules the complete multi-model workload against task-level robotic constraints and metrics.

Second, prior plans are largely fixed once emitted. Design-space exploration, coarse-grained mapping, and offline placement each select an operating point but do not adapt when contention, co-runner mix, or scene complexity changes. On real silicon, those costs do move: overlapped dispatches run $1.137\times$ longer than their solo profiles predict. XPU-RT feeds measured execution back into the runtime and scheduler so deployment can be revised as conditions change.

Third, the deployment unit is usually a single model. Exchange formats, on-device runtimes, and per-network pinning treat each graph independently. A robot running perception, planning, and control at different rates therefore ends up with separate stacks that cannot coordinate end-to-end data age or spend one task’s slack to protect another’s deadline. XPU-RT instead places all models under one schedule with declared rates, dependencies, and deadlines.

Finally, the implementation space is usually fixed before measurement. Autotuners choose among implementations of a fixed operator graph, and schedulers place implementations the compiler has already produced. If the required implementation is absent, additional search cannot recover it. Our largest result makes this concrete: feedback yields a 38.88% improvement below the analytic lower bound of the original implementation graph, which no solver confined to that graph can reach. XPU-RT therefore returns measurement past the scheduler into compilation, so feedback can change what is implementable.

Fine-grained heterogeneous placement is not unique to XPU-RT [11], nor are contention modelling [6] or mission-level evaluation [1]. The distinction is that XPU-RT closes the feedback loop into the implementation space itself and evaluates the consequences full-stack and pre-silicon, with the hardware also treated as a design variable. RoSE provides the underlying evaluation infrastructure, but leaves orchestration and its overhead to the experimenter [23].

3 XPU-RT System Architecture

The design principles of XPU-RT are: First, in order to support the fragmented hardware and software space in robotics, XPU-RT is designed as a modular pipeline such that vendor specific hardware/software components can be easily integrated at multiple levels of abstraction. Secondly, XPU-RT is designed with adaptability; It must be able to emit high quality artifacts for final deployment on robot platforms, as well as low fidelity models and estimates suitable for tightly-integrated co-design loops. Finally, XPU-RT is designed to be lightweight, such that it can be deployed to a broad range of targets, including RTL simulated designs.

XPU-RT’s architecture consists of several phases that operate on multiple levels of abstraction, highlighted in Figure 3. The inputs to XPU-RT are PyTorch representations of robot models and a system specification. Second, XPU-RT lowers each model to an intermediate representation (IR) which is further lowered to an IR consisting of dispatches. Next, XPU-RT provides a code-optimization phase; this phase can either report performance metrics for existing kernels, or execute a full code-optimization loop. Once all dispatches in a graph have been profiled, XPU-RT’s scheduling module binds dispatches to hardware backends and assigns start times to each dispatch. This information is passed to XPU-RT’s runtime generator, which emits an executable for the

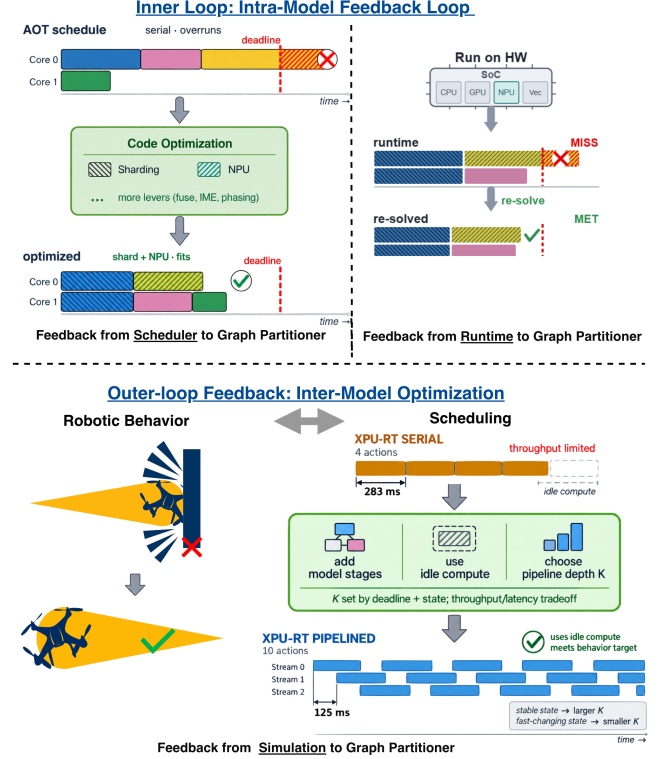


Figure 4. XPU-RT’s inner hardware-feedback loop and outer robot-level feedback loop. Inner loop feedback from generated schedules and hardware performance metrics is used to apply graph transformations within a model, whereas outer loop feedback enables XPU-RT to make changes to the workload specification itself.

target SoC that follows the execution plan set by the scheduler. Finally, XPU-RT provides several evaluation harnesses for kernel and schedule-level performance characterization on hardware, as well as deployment in simulated robotic scenarios.

Each block is designed to be an independent module, communicating through shared interfaces. This design decision enables several key properties; first, blocks can be replaced with wrappers around vendor-provided components as needed (e.g. replacing the code optimization phase with a wrapper around an accelerator’s SDK), and enables modular feedback loops as depicted in Figure 4. This architecture also ensures portability; For example, when migrating between two RISC-V Vector (RVV) SoCs, the same optimized kernels can be used, and only the runtime generator needs to be modified to support the new target’s toolchain.

3.1 System Abstractions

XPU-RT is designed around modular components with well-specified interfaces. Each interface is shared among hardware targets, software SDKs, and robot applications. Starting from a robotic application specification, XPU-RT takes in a

configuration file specifying workloads’ PyTorch definitions, latency and frequency requirements, inter-workload data dependencies, and available hardware backends. This application spec is used to generate an IR describing a directed-acyclic-graph (DAG) of operators, used for all of XPU-RT’s graph optimization phases. XPU-RT-IR interfaces with XPU-RT’s code optimization module via kernel specifications, which define tensor memory layouts, operators, and reference C implementations. Secondly, XPU-RT exports the DAG annotated with feedback-driven profile annotations to XPU-RT’s scheduler. The scheduler emits the same DAG annotated with hardware and timing assignments. Finally, to emit an executable binary, XPU-RT’s runtime generation consumes the scheduled DAG and the kernel specs.

3.2 Graph Extraction and Optimizations

XPU-RT lowers models into a backend-independent IR in which each operator carries its dependencies, shape, precision, quantization, and kernel specification. Tensor records explicitly encode dtype and, for quantized tensors, scale and zero point, while operators carry the corresponding quantization parameters; precision-specific operators such as int8 and fp16 are represented as distinct operator kinds. This lets each backend advertise exactly which numerical formats and operators it supports rather than relying on implicit conversion or fallback.

Graph transformations are exposed through versioned hint contracts for splitting, sharding, fusion, and memory layout, providing explicit hooks for auto-tuning and feedback-driven optimization without coupling the scheduler to compiler internals. Memory layout is a first-class graph property: tensors can be transformed into accelerator-preferred layouts and reshape dispatches inserted only at layout boundaries, allowing regions of the graph to remain in native accelerator layouts rather than repeatedly converting around each kernel. XPU-RT also distinguishes *splitting*, where one dispatch becomes independently schedulable tiles, from *sharding*, where one dispatch executes cooperatively across a multi-core machine combination. Split axes, shard widths, layouts, and implementations can therefore be specialized per operator and backend while preserving the graph’s declared numerical semantics.

3.3 Code Optimization

Code optimization maps each backend-independent kernel specification to a fast, verified implementation for each hardware target. XPU-RT’s code optimizer, ModelBlaster provides XPU-RT’s integrated pipeline for quantization, kernel generation, verification, compilation, and profiling. During graph extraction it supports int8 post-training quantization, fp16 casting, and mixed-precision auto-casting, carrying the selected dtype and quantization parameters into the kernel specification and generated weights. The algorithm database can then select different implementations, data layouts, and

numerical accuracy classes for the same logical operation on different accelerators.

The optimization interface itself is optimizer-agnostic: candidate kernels need only implement the kernel specification and pass the common correctness and measurement pipeline. ModelBlaster can perform agentic optimization through iterative LLM-guided generation and beam search, while external agentic optimizers such as KernelBlaster [7] can operate on the same staged kernel problem using their own search policies and transformation spaces. Numerical correctness remains part of this optimization contract: integer kernels are normally checked bit-exactly, floating-point kernels use dtype-aware tolerances, and backends may declare hardware-specific tolerances when accelerator arithmetic is not bit-reproducible. Thus curated libraries, ModelBlaster’s agentic search, external optimization agents, and vendor SDK implementations can explore performance and precision-specific implementations while exposing the same verified per-dispatch interface to XPU-RT.

3.4 Scheduling

XPU-RT formulates deployment as heterogeneous precedence-constrained scheduling over a single DAG containing all models and their dispatches. Each operation may execute on one of several compatible machine combinations with a profiled execution cost; schedules must satisfy data dependencies and transfer delays, exclusive use of overlapping hardware resources, and release/completion windows for periodic tasks. Periodic networks are expanded into explicit instances with hard timing windows, and the objective lexicographically prioritizes deadline feasibility before minimizing the makespan of best-effort work. XPU-RT supports multiple interchangeable solvers over this formulation, from insertion-based list scheduling and HEFT variants to stochastic search, MILP, and CP-SAT. Heuristic and search backends share a decoder that maps operation priorities to earliest-feasible heterogeneous placements, while CP-SAT directly represents operations as alternative machine intervals with precedence and no-overlap constraints and can be warm-started from fast feasible schedules.

3.5 Runtime Generation

XPU-RT generates a common table-driven runtime from the scheduled graph, with platform-specific execution layers allowing the same scheduling abstraction to target different SoCs. Each dispatch records its implementation, hardware target, dependencies, and start time, while a schedule walker reproduces this ordering through pinned workers, dependency synchronization, and absolute start gates. On embedded RISC-V platforms, XPU-RT builds this layer on an extended Zephyr RTOS, adding hard affinity for heterogeneous harts, per-thread vector-context management, co-processor dispatch, and multi-core execution. Linux-hosted platforms such as SpaceMiT K1 use a POSIX compatibility layer over

the same generated runtime sources, replacing platform-specific threading and timing primitives while retaining the schedule walker. Commercial SoCs use the corresponding hosted execution path and vendor runtime interfaces, such as Qualcomm QNN, while exposing the same profiled-dispatch and scheduled-graph boundaries to the rest of XPU-RT.

The runtime separates *where* an operation executes from *which* implementation executes there, allowing a schedule to select among scalar, vector, and accelerator implementations across heterogeneous resources. For intra-dispatch parallelism, XPU-RT converts multi-core targets into code-generated shard factors executed by a lightweight fixed-width thread pool, with backend-specific weight packing or repacking where required. Activations are shared across backend boundaries according to their IR dtype and layout, while weights and kernel implementations remain backend-specific, preserving accelerator-specific layouts and numerical representations. Platform-specific synchronization, memory fences, vector state management, and timing primitives are contained below this common dispatch abstraction, allowing the generated schedule and upper runtime structure to remain portable across Zephyr-based custom SoCs, Linux SoCs such as K1, and commercial accelerator platforms.

3.6 Runtime Evaluation

XPU-RT evaluates generated workloads across instruction-set simulation, RTL/FPGA emulation, and physical hardware, recording measurement provenance so costs from different fidelity levels remain distinct. Each executable validates its outputs against embedded references while reporting per-dispatch timing and planned-versus-actual traces. Closed-loop studies use the same measurement infrastructure, including measured-latency replay for evaluating how scheduling latency and command rate affect robot behavior. In order to orchestrate these case studies, XPU-RT provides multiple levels of integration. For highest fidelity, XPU-RT integrates with RoSÉ for direct pre-silicon evaluation of SoCs. Secondly, for commercial SoCs, XPU-RT supports hardware-in-the-loop (HIL) integration with robot simulators. For fast iteration, XPU-RT introduces RoSÉ-lite, co-simulating models using host numerics driven by timing annotations extracted from schedule profiles.

3.7 Feedback-driven Co-optimization

SoC Feedback. Offline profiles provide only an approximation of deployed execution. After a schedule is run on the target, XPU-RT compares predicted and measured per-dispatch execution times and uses the resulting trace to recalibrate its cost model. The corrected costs are then fed back into the scheduler, which can either re-solve the existing deployment or re-run the transformation search under the measured hardware behavior. This feedback also distinguishes between misses caused by queueing, which can be addressed by scheduling, and those caused by execution time

alone, which require a different implementation rather than a different placement.

End-to-End Feedback. A schedule that satisfies its local timing constraints may still produce poor closed-loop behavior. XPU-RT therefore evaluates the deployed workload at the robot level, where execution latency determines the rate and freshness of perception, planning, and control outputs. These system-level outcomes provide feedback on whether the timing requirements presented to the scheduler are sufficient for the task. When they are not, the resulting constraints are propagated back through the optimization loop, linking robot-level behavior to scheduling decisions and, when necessary, to compiler transformations that change the available implementation space.

The outer loop evaluates schedules against robot-level behavior rather than hardware timing alone. Common harnesses and sensor interfaces keep model implementations consistent across profiling, baselines, simulation, and hardware, while robot timing requirements map directly to the scheduler’s periods and execution windows. Closed-loop evaluation then measures whether a candidate schedule satisfies the actual robotic objective, allowing system-level outcomes to drive the next compilation and scheduling decision.

4 Experimental Setup

4.1 Hardware and Software Platforms

We evaluate XPU-RT on three heterogeneous SoC families that share the same workload specification, dispatch graph, profile schema, and scheduler, but differ in their execution interface and runtime. This separation tests whether the system abstractions remain unchanged as the underlying hardware and software stack change. Chipyard/FireSim represents customizable pre-silicon hardware: ModelBlaster generates kernels for Saturn RVV and Shuttle+Gemmini targets, which execute under the Zephyr-based XPU-RT runtime with cycle-accurate FPGA timing. SpaceMiT K1 represents a commercial Linux RISC-V SoC with eight X60 harts and an IME matrix engine; the same generated code is cross-compiled for Linux and executed through the POSIX runtime layer, with scheduling across individual harts and multi-hart combinations. Qualcomm QRB5165 represents a driver-attached commercial accelerator SoC: XPU-RT retains graph extraction, profiling, and scheduling, but replaces native code generation with a QNN lane runtime spanning the HTA, DSP, GPU, and CPU. Thus ISA-attached targets expose intra-dispatch sharding and core-width choices, whereas QNN exposes whole-dispatch placement, backend, precision, and graph-partitioning choices.

Across these platforms we evaluate control, perception, and navigation networks both independently and as multi-rate producer-consumer workloads. Our primary real-time metric is the worst response time of the critical model relative to its period; we additionally report deadline misses,

Table 1. Evaluation platforms and the execution choices exposed to XPU-RT.

Platform	Execution stack
SpaceMiT K1	Linux, RVV + IME
QRB5165	Linux, Qualcomm QNN
Chipyard/FireSim (Zephyr RTOS)	
gempair	2 Rocket + Q0.31 Gemmini (no vector unit)
rvvpair	2 Rocket + Saturn RVV (no Gemmini)
hetero	1 Gemmini + 1 RVV hart
quad	2 Gemmini + 2 RVV harts

makespan, schedule throughput, and predicted-versus-measured execution time where the platform exposes sufficiently precise timing. Closed-loop experiments then translate these scheduling outcomes into robot-level task success and control rate.

5 Evaluation

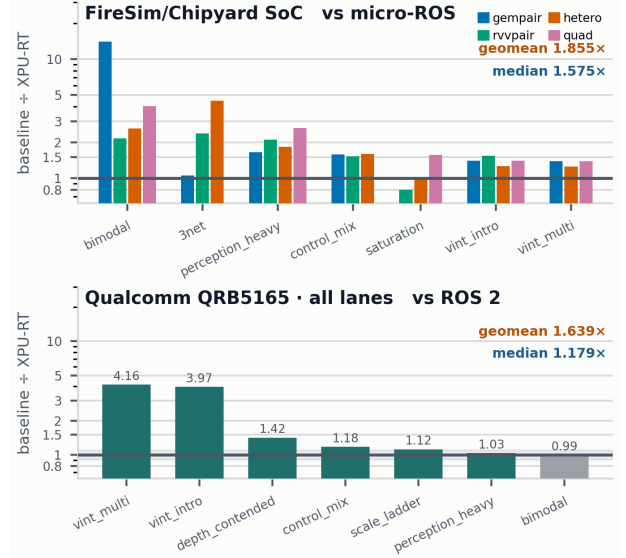
5.1 Model and Workload Selection

We select models that provide adequate coverage of scheduler-relevant properties such as dispatchable operator count, graph structure, precision, and backend affinity. The suite ranges from 7 to 605 operators and includes short control heads, convolutional perception models, producer/consumer chains, and attention/GEMM-heavy models. Precision is also heterogeneous: most models use INT8, MLP-CONTROL remains FP32, and models such as VINT contain mixed-precision execution, forcing the scheduler and cost model to reason about different kernel families within the same workload. Only components actually deployed on-device are counted. Controlled model rungs vary width or input resolution while preserving topology, isolating compute scaling from changes in the placement graph.

We compose these models into eleven workloads spanning rate diversity, cross-network precedence, contention, oversubscription, deadline pressure, and heterogeneous operator mixes. The hardware sweep is target-specific: on FireSim, we vary performance/efficiency worker configurations from the same bitstream and compare unsharded against 2-way-sharded execution; on Qualcomm, placement spans the CPU, GPU, DSP, and HTA; and on SpaceMiT K1, scheduling chooses between the vector and matrix clusters. These axes expose the target-specific placement and parallelism choices available to XPU-RT while keeping the workload specification and scheduling abstraction unchanged.

5.2 XPU-RT against ROS

A widely used runtime and middleware in robotics is the Robot Operating System (ROS). We evaluate against ROS2, a revision with a transport layer supporting real-time features. We evaluate two use cases: On the QRB5165 we target the full ROS2 Foxy platform targeting Linux, while on the

**Figure 5.** XPU-RT versus fixed ROS/ μ ROS deployment on commercial and pre-silicon heterogeneous platforms.

Zephyr-based FireSim targets we use μ ROS Jazzy, a lightweight distribution for embedded targets. To perform this evaluation, we extend μ ROS executor with new multicore support leveraging Zephyr RTOS tasks. Because ROS models workloads as a series of distributed nodes, it does not have a global shared schedule across hardware backends; instead, we pin each model to a node bound to a hardware backend, using the best performing configuration. A summary of our results is depicted in Figure 5.

5.3 XPU-RT’s Inner Feedback Loop

On a repeating 100 ms K1 workload of 14 jobs and 178 dispatches, feedback reduces the globally optimal worst critical-model response from **8.00 ms to 4.89 ms (38.88%)**. This is a separation in implementation space rather than simply a better schedule: the original schedule attains an 8.00 ms lower bound computed from the fastest legal implementation of every dispatch under unlimited resources, while the feedback schedule attains a new 4.89 ms bound. Because the latter lies below the original graph’s lower bound, no scheduler, including an exhaustive oracle, could reach it without changing the available implementations. Feedback exposes measured multi-hart sharded implementations while holding the models, timing requirements, hardware, objective, and profile corpus fixed.

Real-board execution preserves this advantage. Across ten runs per schedule, median worst response falls from **10.49 ms to 7.20 ms (31.29%)**, with every feedback run faster than every original run ($p = 5.41 \times 10^{-6}$). Figure 6 illustrates the same mechanism on a second K1 taskset. First, an operator whose single-hart latency exceeds its deadline cannot be repaired by placement alone; feedback requests a sharded

Table 2. Evaluation models and workload families, providing coverage for model architectures and deployment scenarios.

	Model	Ops	Prec.	Role
Models	MLP-CONTROL	7	FP32	periodic control
	DRONET	21	INT8	navigation CNN
	FASTDEPTH	114	INT8	depth producer
	YOLOV8-NANO	155	INT8	object detection
	VINT	605	Mixed	transformer navigation
	OCTO-SMALL-1.5	21–71	INT8	generalist robot policy
	SMOLVLA	49	INT8	vision–language–action policy
	Family	Models	Primary stressor	
Workload families	BIMODAL	YOLOV8-NANO, MLP-CONTROL	long job + fast control	
	CONTROL_MIX	MLP-CONTROL, DRONET, YOLOV8-NANO	mixed rates	
	DEPTH_CHAIN	FASTDEPTH, DRONET	producer–consumer precedence	
	DEPTH_CONTENTENDED	FASTDEPTH, DRONET, YOLOV8-NANO	precedence + contention	
	PERCEPTION_HEAVY	YOLOV8-NANO, MLP-CONTROL	detection-heavy load	
	SATURATION	YOLOV8-NANO, DRONET, MLP-CONTROL	oversubscription	
	SCALE_LADDER	DRONET	model scaling	
	TIGHT_LOOP_HOT	MLP-CONTROL, DRONET	deadline pressure	
	VINT_INTRO	VINT, MLP-CONTROL	transformer + control	
	VINT_MULTI	VINT, DRONET, MLP-CONTROL	heterogeneous operator mix	

implementation and eliminates two deadline misses. Second, the resulting schedule meets all predicted deadlines but runs about 31% slower on silicon, causing four misses; returning the measured costs and re-solving restores feasibility without changing the workload. Real-board traces also show contention-induced cost drift, further motivating measured-cost recalibration rather than relying solely on offline profiles.

5.4 XPU-RT’s Outer Feedback Loop via End-to-End Manipulation on QRB5165

We use manipulation to test whether SoC-level scheduling decisions propagate to robot-level behavior. The workload is the on-device INT8 portion of Octo-small-1.5 (27.0 M parameters), scheduled on a Qualcomm QRB5165 across CPU, DSP, and HTA; the fixed-language embedding is precomputed off-device. Measured board timing is replayed to a robotic simulator (SIMPLER), so the QRB5165 is not live in the control loop, but each evaluated operating point corresponds to a schedule measured on the physical SoC.

We evaluate two robot embodiments with substantially different actuation rates. The WidowX 250 S consumes a new command every 40 ms and is evaluated on EGGPLANT IN BASKET and SPOON ON TOWEL. The Google Everyday Robots arm consumes commands only every 333 ms and is evaluated on PICK COKE CAN and CLOSE DRAWER. This distinction matters: the WidowX can consume essentially every improvement in policy cadence, whereas a fast schedule can produce multiple actions between two Google-robot actuation ticks, after which additional throughput is discarded.

From the robot’s perspective, each schedule exposes two distinct quantities: *observation latency*, the age of the observation underlying an action, and *release period*, the interval

between successive fresh actions. Pipelining places these objectives in tension: improving cadence can increase the latency of an individual inference.

Figure 7 separates placement from pipelining. The vendor baseline executes the model on the CPU while DSP and HTA remain idle, producing an action every 677 ms. Heterogeneous placement alone lowers the serial interval to 250 ms (2.7×), but a single inference remains a dependency chain and cannot fill all three engines simultaneously. Pipelining instead overlaps different inferences across the idle regions, producing an action every 144 ms—4.7× the vendor rate and a further 1.7× beyond placement alone. The per-inference latency improves by only 2.4× (677 to 283 ms), emphasizing that throughput and latency are different optimization objectives. The kernels and per-inference work are unchanged; the gain comes from exposing otherwise idle heterogeneous resources to the scheduler.

The closed-loop sweep shows that the hardware gain matters only when the robot can use it. On the WidowX, the CPU baseline succeeds on only 1.7% of eggplant and 1.4% of spoon episodes, versus 52.4% and 43.8% for a fast pipelined schedule. Across the full 44-point schedule plane, success is strongly associated with release period ($\rho = -0.830$ and -0.843 for eggplant and spoon) but only weakly with latency. A particularly revealing schedule has worse observation latency than serial execution (385 versus 283 ms) but much faster cadence (111 versus 283 ms), raising eggplant success from 19.3% to 45.4%. For these tasks, providing fresh actions more frequently matters more than minimizing the completion time of any one inference.

The Google robot exposes the opposite regime. Its 333 ms actuation grid already saturates under most competitive schedules, so additional actions are discarded. Accordingly,

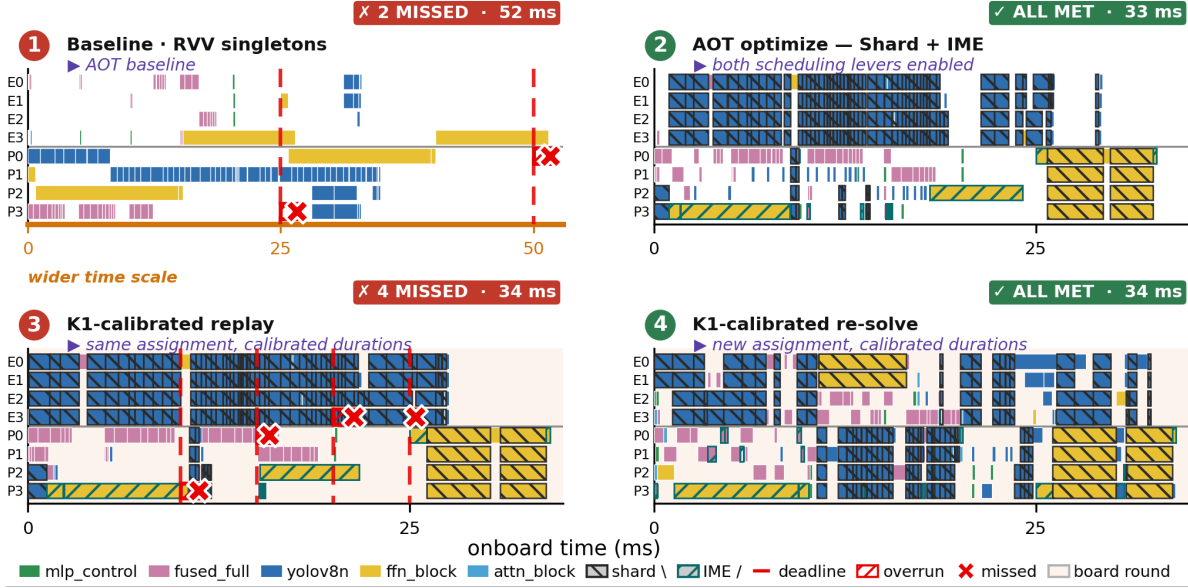


Figure 6. Feedback changes implementations and schedules on K1. ① The RVV baseline misses two deadlines. ② Sharding and IME placement eliminate them. ③ Measured board costs introduce four misses. ④ Re-solving with those costs restores feasibility.

CLOSE DRAWER is statistically unchanged across the schedule ladder (40.0% to 36.8%, all paired $p > 0.06$), while PICK COKE CAN exhibits a weaker, non-monotonic response. Thus the benefit is not a property of model speed alone: it depends on whether the downstream robot can consume the additional inference throughput.

This contrast motivates XPU-RT’s outer feedback loop. Across the feasible schedule plane, latency and period are negatively correlated ($\rho = -0.62$), and the task-specific optimum moves along that frontier: the WidowX tasks favor periods near 130 ms, whereas the Google-robot tasks peak at substantially slower 200–283 ms operating points. Consequently, neither minimum latency, minimum period, nor a fixed weighting of the two can select the best deployment across tasks. XPU-RT instead evaluates feasible schedules against the robot-level objective and uses that outcome to select the operating point. The same result holds for energy: on the two WidowX tasks the slower schedules consume 1.71× and 1.42× more modeled actuator energy, largely because failed episodes run longer, so improved task success is not purchased by increased energy.

5.5 Closed-loop co-simulation: system-level impact

We next ask whether the timing improvements above change robot behavior. We fly a Crazyflie through a four-gate Isaac Lab warehouse course using the deployed K1 workload: CTRL (mlp_control) at 100 Hz, NAV (fused_full) at 50 Hz, and YOLO (yolov8_nano) for perception. We use RoSÉ-lite to instrument the timing of the schedule into the simulator. This deterministic replay converts the measured schedule

directly into command latency. We additionally sweep cruise speed and control rate to characterize the resulting flight envelope.

The control loop runs on a 10 ms tick. XPU-RT’s feedback-sharded schedule responds in 4.89 ms and sustains the deployed 100 Hz, whereas ROS pinning responds in 12.40 ms, misses the tick, and drops to 50 Hz. The two flight arms otherwise use identical hardware, networks, controller, course, and cruise speed. XPU-RT also shards and pipelines YOLO across the K1 cores, holding perception near its 22 ms budget, while the static ROS partition backs up and starves control. Over six matched seeds, XPU-RT completes the course in 3/6 runs while ROS completes 0/6; the representative ROS flight clears the first gate before losing stability and crashing, whereas XPU-RT clears all four gates.

The broader sweep confirms that the representative flight is not an isolated outcome. Across 240 flights, success rises sharply around a 50 Hz control-rate floor ($p < 0.001$); the same transition appears on a second, unseen gate course without retuning (+23 pts, $p = 0.02$). Thus the same silicon and workload can either remain stable or crash based on the command rate sustained by the schedule.

Scheduling is not the only lever. It reduces the control-loop response from 12.40 to 4.89 ms and restores the required 100 Hz, but YOLO remains compute-bound at roughly an 18 ms critical path. Raising perception rate further requires the code-generation change of \$5.3, which routes convolution work to the IME (1.67×, four deadline misses to none).

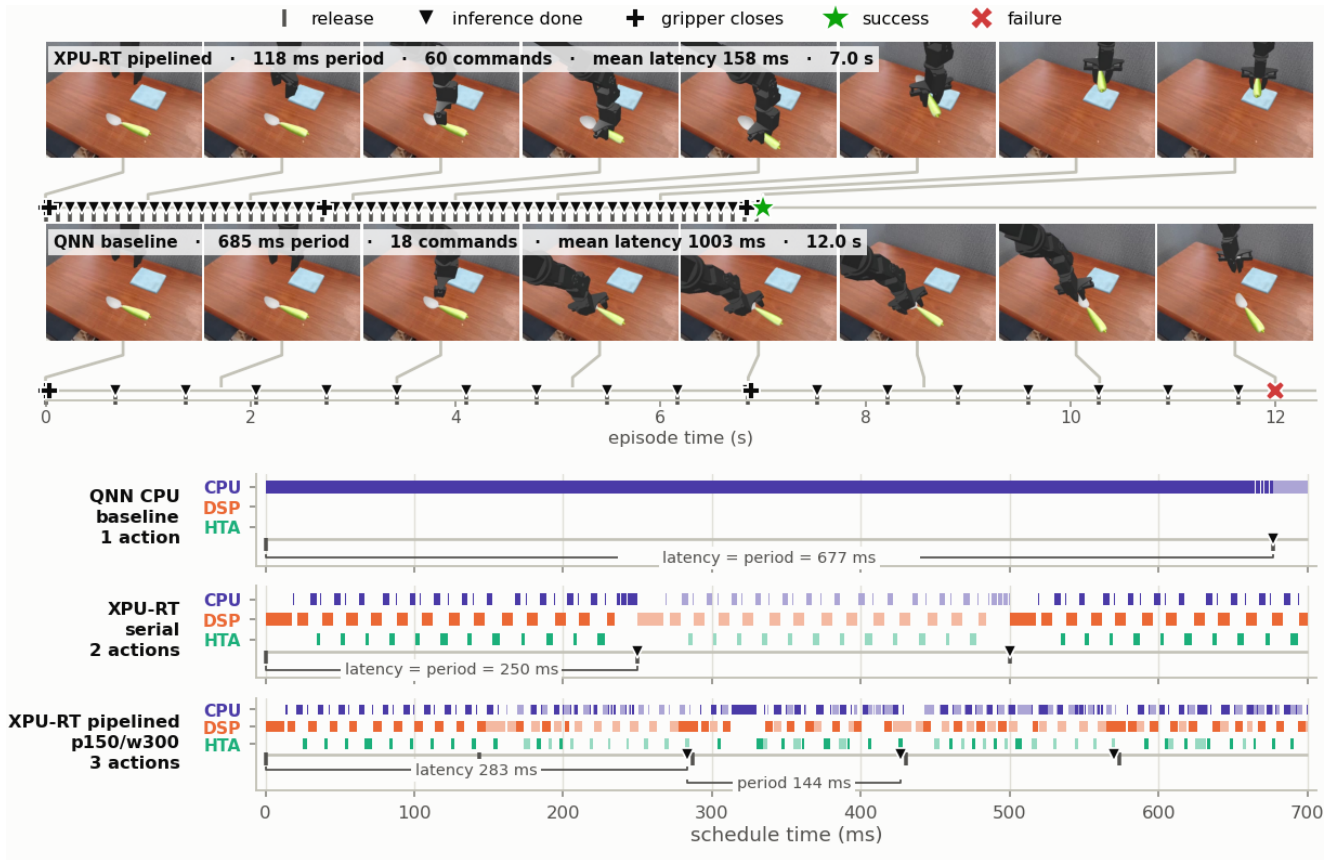


Figure 7. Placement and pipelining increase QRB5165 action rate. CPU-only QNN produces an action every 677 ms; heterogeneous placement reduces this to 250 ms and pipelining to 144 ms (4.7 $\times$), while one inference spans 283 ms.

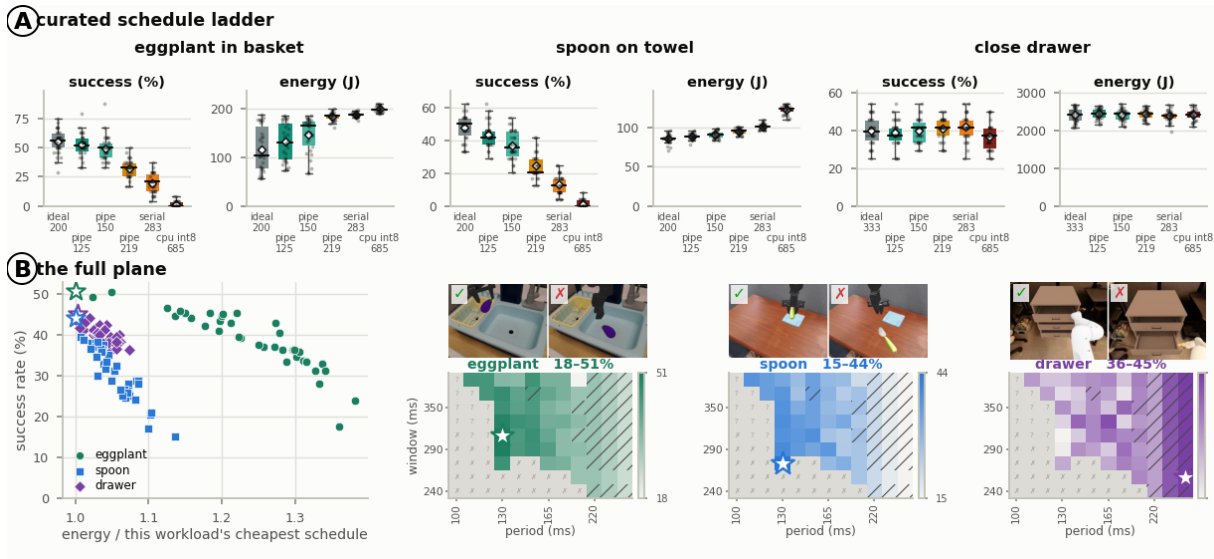


Figure 8. Robot-level outcomes depend on schedule choice. (A) Success and modeled actuator energy across representative schedules. (B) The 44-point CP-SAT sweep over energy, release period, and latency; stars mark best schedules and $\times$ / $?$ infeasible/unresolved points.

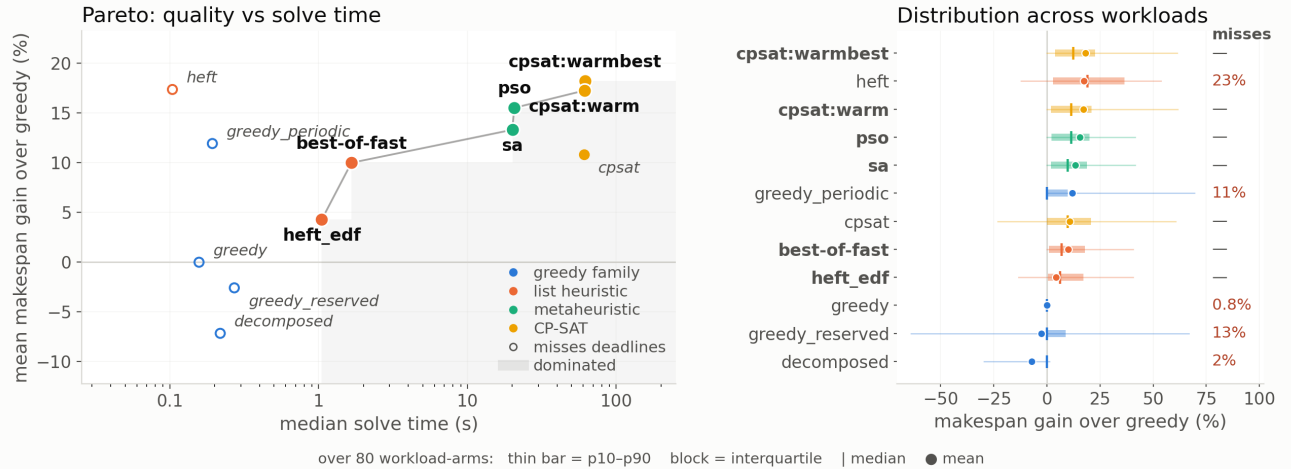


Figure 9. Solver choice trades modest schedule quality for offline solve time. Among deadline-feasible solutions, a fast portfolio captures most of the available gain in under a second, while warm-started CP-SAT reaches the best quality at higher cost. Hollow markers denote deadline violations.

5.6 Scheduler Solver Design Space Exploration

The gains above do not depend on an unusually expensive solver. We sweep solver choices across all of our workloads in Table 2 and compare schedule quality against offline solve time. Deadline feasibility is enforced first: a shorter makespan is not useful if periodic windows are missed.

Among deadline-feasible solvers, the Pareto frontier is narrow: best-of-fast, a policy that aggregates the policy among all sub-second solvers, improves makespan by 7.6% in about 0.5 s, pso reaches 9.1% in roughly 11 s, and warm-started CP-SAT reaches 10.0% in about 15 s. Thus spending 30× more solve time than the fast portfolio buys only 2.4 percentage points of additional improvement. Cold CP-SAT is both slower and worse, showing that warm starting, rather than CP-SAT alone, makes the exact solver competitive.

Solver choice therefore affects how efficiently XPU-RT searches a given implementation space, but it does not explain the larger gains reported in §5.3. There, feedback produces a schedule below the lower bound of the original implementation graph, a region no solver confined to that graph can reach. The dominant benefit comes from changing what can be implemented; stronger solvers provide a smaller, secondary improvement within that space.

6 Conclusion

Robotic deployment on heterogeneous SoCs is not simply a kernel-optimization or placement problem. A useful deployment must coordinate multiple models with different rates and deadlines, account for the platform-specific forms of parallelism each SoC exposes, and ultimately preserve the timing properties that matter to the robot. XPU-RT addresses this problem with a common full-stack abstraction spanning compilation, heterogeneous scheduling, runtime execution,

and closed-loop evaluation, while retaining platform-specific optimization opportunities beneath that interface.

The central result is that feedback must reach beyond selecting a better point in a fixed design space. On K1, feedback produces a 4.89 ms critical-model response that lies below the 8.00 ms lower bound of the original implementation graph, demonstrating that the improvement comes from changing what can be implemented rather than searching the same space harder. On QRB5165, heterogeneous placement and pipelining increase policy action rate 4.7×, and closed-loop manipulation shows that the resulting cadence changes both task success and modeled actuator energy when the robot can consume it. In flight, the same principle raises the sustainable control rate from 50 to 100 Hz relative to ROS 2 and changes the outcome from repeated loss of control to successful course completion. Together, these results show that the right deployment cannot be chosen from hardware timing alone: hardware measurements must shape the available implementations and schedules, and robot behavior must determine which of those operating points is actually useful.

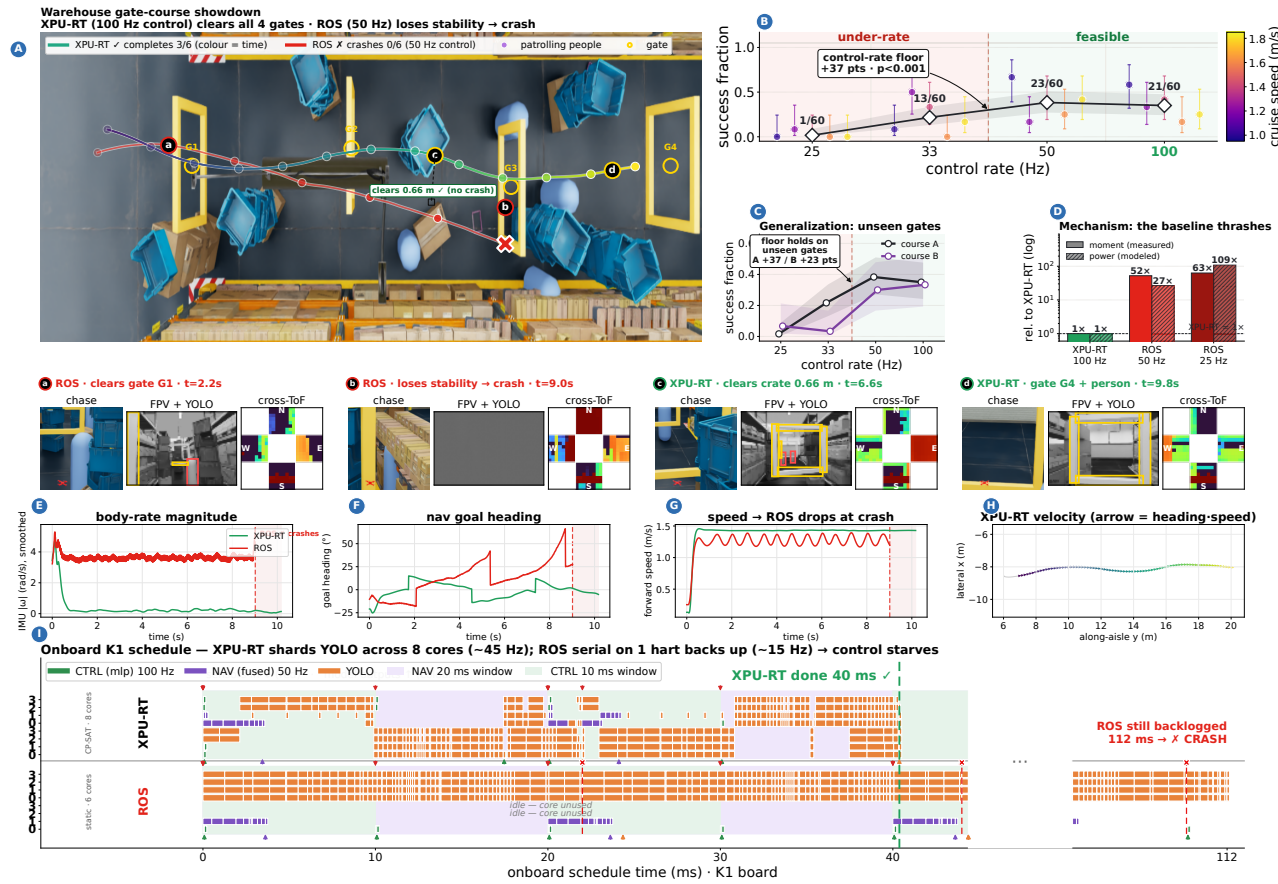


Figure 10. Schedule-level improvements change closed-loop flight behavior. Under the same K1 workload, controller, course, and cruise speed, XPU-RT sustains 100 Hz control and completes 3/6 matched runs, while ROS falls to 50 Hz and completes 0/6. The flight-envelope sweep (240 flights) reveals a control-rate floor near 50 Hz (+37 pts from 25 to 50 Hz, $p < 0.001$), which reproduces on an unseen course (+23 pts, $p = 0.02$). Below this floor the rate-starved baseline oscillates and commands $\sim 50\times$ larger body moments. The schedule trace shows the underlying cause: XPU-RT shards and pipelines perception while the ROS partition accumulates backlog and starves control.

References

- [1] Behzad Boroujerdian, Hasan Genc, Srivatsan Krishnan, Wenzhi Cui, Aleksandra Faust, and Vijay Janapa Reddi. 2018. MAVBench: Micro Aerial Vehicle Benchmarking. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 894–907. doi:10.1109/MICRO.2018.00077
- [2] Behzad Boroujerdian, Radhika Ghosal, Jonathan Cruz, Brian Plancher, and Vijay Janapa Reddi. 2021. RoboRun: A Robot Runtime to Exploit Spatial Heterogeneity. In *Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC)*. 829–834. doi:10.1109/DAC18074.2021.9586280
- [3] Daniel Casini, Tobias Blaß, Ingo Lütkebohle, and Björn B. Brandenburg. 2019. Response-Time Analysis of ROS 2 Processing Chains Under Reservation-Based Scheduling. In *31st Euromicro Conference on Real-Time Systems (ECRTS 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 133)*. 6:1–6:23. doi:10.4230/LIPIcs.ECRTS.2019.6
- [4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594. doi:10.5555/3291168.3291211
- [5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *arXiv preprint arXiv:1410.0759* (2014). arXiv:1410.0759
- [6] Ismet Dagli and Mehmet E. Belviranli. 2024. Shared Memory-Contention-Aware Concurrent DNN Execution for Diversely Heterogeneous System-on-Chips. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 243–256. doi:10.1145/3627535.3638502
- [7] Shengjun Kris Dong, Sahil Modi, Dima Nikiforov, Sana Damani, Edward Lin, Siva Kumar Sastry Hari, and Christos Kozyrakis. 2026. KernelBlaster: Continual Cross-Task CUDA Optimization via Memory-Augmented In-Context Reinforcement Learning. *arXiv preprint arXiv:2602.14293* (2026). arXiv:2602.14293
- [8] Shengjun Kris Dong, Dima Nikiforov, Widyadewi Soedarmadji, Minh Nguyen, Vikram Jain, Christopher W. Fletcher, and Yakun Sophia Shao. 2025. Characterizing and Optimizing Real-Time Optimal Control for Embedded SoCs. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*. 489–503. doi:10.1109/IISWC66894.2025.00047
- [9] Google LLC. 2019. XNNPACK. <https://github.com/google/XNNPACK>.
- [10] Claire Le Goues, Sebastian Elbaum, David Anthony, Z. Berkay Celik, Mauricio Castillo-Effen, Nikolaus Correll, Pooyan Jamshidi, Morgan Quigley, Trenton Tabor, and Qi Zhu. 2024. Software Engineering for Robotics: Future Research Directions; Report from the 2023 Workshop on Software Engineering for Robotics. arXiv:2401.12317 [cs.RO] <https://arxiv.org/abs/2401.12317>
- [11] Mingcong Han, Weihang Shen, Rong Chen, Binyu Zang, and Haibo Chen. 2025. Holistic Heterogeneous Scheduling for Autonomous Applications using Fine-grained, Multi-XPU Abstraction. *arXiv preprint arXiv:2508.09503* (2025). arXiv:2508.09503 doi:10.48550/arXiv.2508.09503
- [12] Yuka Ikarashi, Kevin Qian, Samir Droubi, Alex Reinking, Gilbert Louis Bernstein, and Jonathan Ragan-Kelley. 2025. Exo 2: Growing a scheduling language. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 426–444. doi:10.1145/3669940.3707218
- [13] Seah Kim, Hyoukjun Kwon, Jinook Song, Jihyuck Jo, Yu-Hsin Chen, Liangzhen Lai, and Vikas Chandra. 2023. DREAM: A Dynamic Scheduler for Dynamic Real-time Multi-model ML Workloads. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 4*. doi:10.1145/3623278.3624753
- [14] Srivatsan Krishnan, Zishen Wan, Kshitij Bhardwaj, Paul Whatmough, Aleksandra Faust, Sabrina Neuman, Gu-Yeon Wei, David Brooks, and Vijay Janapa Reddi. 2022. Automatic Domain-Specific SoC Design for Autonomous Unmanned Aerial Vehicles. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. doi:10.1109/MICRO56248.2022.00033
- [15] Liu Liu, Jie Tang, Shaoshan Liu, Bo Yu, Yuan Xie, and Jean-Luc Gaudiot. 2021. II-RT: A Runtime Framework to Enable Energy-Efficient Real-Time Robotic Vision Applications on Heterogeneous Architectures. *Computer* 54, 4 (2021), 14–25. doi:10.1109/MC.2020.3015950
- [16] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. 2022. Robot Operating System 2: Design, Architecture, and Uses in the Wild. *Science Robotics* 7, 66 (2022), eabm6074. doi:10.1126/scirobotics.abm6074
- [17] Víctor Mayoral-Vilches, Jason Jabbour, Yu-Shun Hsiao, Zishen Wan, Martín Crespo-Álvarez, Matthew Stewart, Juan Manuel Reina-Muñoz, Prateek Nagras, Gaurav Vikhe, Mohammad Bakhshalipour, Martin Pinzger, Stefan Rass, Smruti Panigrahi, Giulio Corradi, Niladri Roy, Phillip B. Gibbons, Sabrina M. Neuman, Brian Plancher, and Vijay Janapa Reddi. 2024. RobotPerf: An Open-Source, Vendor-Agnostic, Benchmarking Suite for Evaluating Robotics Computing System Performance. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. 8288–8297. doi:10.1109/ICRA57147.2024.10610841
- [18] Justin McGowen, Ismet Dagli, Neil T. Dantam, and Mehmet E. Belviranli. 2024. Scheduling for Cyber-Physical Systems with Heterogeneous Processing Units under Real-World Constraints. In *Proceedings of the 38th ACM International Conference on Supercomputing (ICS)*. ACM, 298–311. doi:10.1145/3650200.3656625
- [19] MOSEK ApS. 2024. The MOSEK Optimization Suite. <https://docs.mosek.com/>.
- [20] Mergen Nachin, Digant Desai, Sicheng Stephen Jia, Chen Lai, Mengwei Liu, Jacob Szwejbka, Razi Alvaraz, RJ Ascani, Dave Bort, Manuel Candales, Andrew Caples, Yanan Cao, Zhengxu Chen, Soumith Chintala, Gregory Comer, Tanvir Islam, Songhao Jia, Tarun Karuturi, Jack Khuu, Abhinay Kukkadapu, Tugsbaysgalan Manlaibaatar, Andrew Or, Kimish Patel, Siddhartha Pothapragada, Lucy Qiu, Supriya Rao, Orion Reblitz-Richardson, Max Ren, Scott Roy, Anthony Shoumikhin, Scott Wolchok, Guang Yang, Angela Yi, Martin Yuan, Hansong Zhang, Jack Zhang, Jerry Zhang, Shunting Zhang, and C. Cagatay Bilgin. 2026. ExecuTorch: A Unified PyTorch Solution to Run AI Models On-Device. arXiv:2605.08195 [cs.LG]
- [21] Sabrina M. Neuman, Radhika Ghosal, Thomas Bourgeat, Brian Plancher, and Vijay Janapa Reddi. 2023. RoboShape: Using Topology Patterns to Scalably and Flexibly Deploy Accelerators Across Robots. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*. doi:10.1145/3579371.3589104
- [22] Sabrina M. Neuman, Brian Plancher, Thomas Bourgeat, Thierry Tambe, Srinivas Devadas, and Vijay Janapa Reddi. 2021. Robomorphic Computing: A Design Methodology for Domain-Specific Accelerators Parameterized by Robot Morphology. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 674–686. doi:10.1145/3445814.3446746
- [23] Dima Nikiforov, Shengjun Kris Dong, Chengyi Lux Zhang, Seah Kim, Borivoje Nikolić, and Yakun Sophia Shao. 2023. RoSE: A Hardware-Software Co-Simulation Infrastructure Enabling Pre-Silicon Full-Stack Robotics SoC Evaluation. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*. 1–15. doi:10.1145/3579371.3589099
- [24] Mohanad Odema, Luke Chen, Hyoukjun Kwon, and Mohammad Abdullah Al Faruque. 2024. SCAR: Scheduling Multi-Model AI Workloads on Heterogeneous Multi-Chiplet Module Accelerators. In *Proceedings*

- of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 565–579. doi:10.1109/MICRO61859.2024.00049
- [25] ONNX Project Contributors. [n. d.]. Open Neural Network Exchange (ONNX). <https://onnx.ai/>.
- [26] ONNX Runtime developers. 2018. ONNX Runtime. <https://onnxruntime.ai>.
- [27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*.
- [28] Laurent Perron, Frédéric Didier, and Steven Gay. 2023. The CP-SAT-LP Solver (Invited Talk). In *29th International Conference on Principles and Practice of Constraint Programming (CP) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 280)*. 3:1–3:2. doi:10.4230/LIPIcs.CP.2023.3
- [29] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 519–530. doi:10.1145/2491956.2462176
- [30] Joseph Rogers, Lieven Eeckhout, and Magnus Jahre. 2025. HILP: Accounting for Workload-Level Parallelism in System-on-Chip Design Space Exploration. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1275–1288. doi:10.1109/HPCA61900.2025.00097
- [31] Mohammad Siavashi, Faezeh Keshmiri Dindarloo, Dejan Kostic, and Marco Chiesa. 2025. Priority-Aware Preemptive Scheduling for Mixed-Priority Workloads in MoE Inference. In *Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys)*. ACM, 132–138. doi:10.1145/3721146.3721956
- [32] Arne Symons, Linyan Mei, Steven Coleman, Pouya Houshmand, Sebastian Karl, and Marian Verhelst. 2025. Stream: Design space exploration of layer-fused DNNs on heterogeneous dataflow accelerators. *IEEE Trans. Comput.* 74, 1 (2025), 237–249. doi:10.1109/TC.2024.3477938
- [33] The IREE Authors. 2019. IREE: An MLIR-based Compiler and Runtime for ML Models from Multiple Frameworks. <https://github.com/iree-org/iree>. <https://iree.dev/>.
- [34] Steve Vestal. 2007. Preemptive Scheduling of Multi-criticality Systems with Varying Degrees of Execution Time Assurance. In *Proceedings of the 28th IEEE International Real-Time Systems Symposium (RTSS)*. 239–243. doi:10.1109/RTSS.2007.47
- [35] Zishen Wan, Ashwin Lele, Bo Yu, Shaoshan Liu, Yu Wang, Vijay Janapa Reddi, Cong Hao, and Arijit Raychowdhury. 2022. Robotic Computing on FPGAs: Current Progress, Research Challenges, and Opportunities. In *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. 291–295. doi:10.1109/AICAS54282.2022.9869951
- [36] Zishen Wan, Bo Yu, Thomas Yuang Li, Jie Tang, Yuhao Zhu, Yu Wang, Arijit Raychowdhury, and Shaoshan Liu. 2020. A Survey of FPGA-Based Robotic Computing. *IEEE Circuits and Systems Magazine* 21, 2 (2020), 48–74.
- [37] Yuanfan Xu, Jincheng Yu, Suquan Zhang, Yunfei Xiang, Hongyang Jia, and Yu Wang. 2024. Invited: Automatic Hardware/Software Design for High-Speed Autonomous Unmanned Aerial Vehicles Guided by a Flight Model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. ACM, 1–4. doi:10.1145/3649329.3663516
- [38] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen,
- Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879. <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [39] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. 2020. FlexTensor: An Automatic Schedule Exploration and Optimization Framework for Tensor Computation on Heterogeneous System. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 859–873. doi:10.1145/3373376.3378508
- [40] An Zou, Yuankai Xu, Yinchen Ni, Jintao Chen, Yehan Ma, Jing Li, Christopher Gill, Xuan Zhang, and Yier Jin. 2025. A Survey of Real-time Scheduling on Accelerator-based Heterogeneous Architecture for Time Critical Applications. *arXiv preprint arXiv:2505.11970* (2025). doi:10.48550/arXiv.2505.11970